

AFTER HOURS CAFE

Game Design & Modular Systems Specification

MAKE COFFEE. BUILD A PLACE. KNOW YOUR PEOPLE.

A cozy first-person night-cafe game about satisfying coffee craft, recurring customers, social discovery, and turning a small rented shop into a place you truly own.

Platform	PC / Steam
Engine	Unity - URP
Target Scope	Solo-developed, ~3 months to polished first release

Design rule: small systems, strong interactions, visible consequences.

1. Product Vision

Elevator Pitch

A cozy first-person night cafe in a Korean-inspired neighborhood. The player manually crafts coffee, serves recurring customers, shapes the cafe identity through decor, watches customers share experiences on an in-game social network, unlocks helpful robots, and grows a struggling rented shop into a place they can finally buy and name.

1.1 Player Fantasy

- I own a tiny warm place that feels safe against the rainy city outside.
- I know my regulars, their usual drinks, habits, and small life stories.
- Coffee making feels tactile, fast, readable, and satisfying - never like menu work.
- My design choices change who visits, what they post, and how the cafe develops.
- I gradually transform an empty rental into a personal, memorable place.

1.2 Design Pillars

Pillar	Rule	Player Result
Tactile Craft	Coffee actions stay manual, short, readable, and juicy.	Making every drink feels good.
Living Customers	Use a limited cast of recurring NPCs with memory and preferences.	Players recognize people, not spawn IDs.
Cozy Contrast	Cold/rainy exterior vs warm interior is the visual identity.	Screenshots feel instantly recognizable.
Meaningful Decor	Furniture carries style/function tags and affects visitors.	Decoration is gameplay, not only cosmetics.
Organic Discovery	Social posts and word-of-mouth create demand naturally.	The world generates goals without quest spam.
Assist, Never Replace	Robots remove chores but do not automate core coffee craft.	Progress adds comfort without deleting gameplay.
Visible Growth	Cafe, neighborhood, memory wall, customers and tools visibly evolve.	Money and time always feel meaningful.

1.3 Scope Guardrails

Version 1 target

One cafe, one small exterior street, one shift structure, 8-12 drinks, 12-20 recurring customers, 20-40 decor items, 2 support robots, one cat, compact campaign arc, then endless mode.

- No multiplayer.
- No open world.
- No realistic accounting simulation.
- No deep employee management.
- No generative-AI NPC dialogue in v1.
- No full restaurant cooking system.
- No multiple cafe branches.

2. Core Experience & Game Loop

2.1 Shift Loop

Phase	Player Actions	Systems Active
Pre-Open	Review phone, choose music, restock, adjust decor/menu.	Phone, inventory, decor, music, economy
Open	Take orders, make drinks, serve, clean, observe customers.	Orders, coffee, customers, seating, tray, cleaning
Living Cafe	Customers interact, post, create trends, trigger small events.	Relationships, CafeGram, events, ambience
Close	Flip sign, finish last tasks, see summary, feed cat, reflect.	Shift state, stats, memories, economy
Progress	Buy upgrades/decor, unlock recipes/robots, expand identity.	Shop, progression, reputation, ownership

2.2 Long-Term Campaign Arc

Primary Goal

The cafe begins as a struggling rental. The campaign ends when the player reaches the required money + reputation milestone to buy the cafe. On purchase, the player permanently names it and unlocks Endless Mode.

- Early game: survive comfortably, learn 3-5 recipes, meet first regulars.
- Mid game: define cafe identity, unlock social reach, first robot, signature drinks, events.
- Late game: premium upgrades, strong regular base, neighborhood changes, ownership fund.
- Post-campaign: Endless Mode with open progression, collections, seasons/events and optimization.

2.3 Player Motivation Stack

Motivation	What Advances It	What It Unlocks
Money	Sales, tips, trends, efficient service	Equipment, furniture, ingredients, expansion, ownership
Reputation	Good service, posts, repeat visits, special nights	Higher-tier customers, items, events
Relationships	Repeat visits, remembered orders, successful service	Usual-order shortcut, story beats, memories
Identity	Decor/style tags, menu choices, music, layout	Customer archetypes, decor families, vibes
Collection	Recipes, records, Polaroids, cups, milestones	Completion goals + visual history
Ownership	Money + reputation + campaign milestones	Cafe purchase, custom permanent name, Endless Mode

3. Technical Architecture Principles

Architecture Goal

Build every major mechanic as an independent feature module. Modules communicate through contracts/events, not direct scene references. A module should be testable in isolation and replaceable without rewriting unrelated systems.

3.1 Unity Conventions

- Data-first: immutable/static definitions in ScriptableObjects; runtime state in plain serializable C# models.
- Event-driven: publish domain events through a central EventBus or typed signal layer.
- Interfaces at boundaries: e.g., IInteractable, IOrderSource, IOrderConsumer, ICustomerService, ISaveable.
- State machines for shift flow, customer behavior, order lifecycle, robot behavior and interaction tools.
- Dependency injection/service registry for cross-cutting services; avoid FindObjectOfType and hidden singletons.
- Scene composition only: scene objects reference local presenters/controllers, never own global game truth.
- Save schema is versioned from day one; systems serialize only their own state.
- Debug commands and deterministic seeds where useful for repeatable system tests.

3.2 Data Layers

Layer	Examples	Persistence
Definition Data	DrinkDefinitionSO, IngredientSO, DecorItemSO, CustomerProfileSO, EventDefinitionSO	Project assets - never saved per run
Runtime State	Current order, customer FSM state, active weather, open shift	Memory only / reconstructable
Persistent State	Money, reputation, relationships, owned items, unlocks, memories	SaveGame JSON/binary snapshot
Analytics/Debug	Event traces, economy logs, order timings, errors	Optional local/dev logging

3.3 Core Domain Events

Event	Typical Publishers	Typical Subscribers
ShiftOpened / ShiftClosed	ShiftSystem	CustomerSpawner, Music, Economy, Summary UI
OrderCreated / Updated / Completed / Failed	OrderSystem	HUD, Customer, Economy, Analytics
DrinkCrafted	CoffeeCrafting	OrderSystem, RecipeProgress, Analytics
CustomerEntered / Seated / Served / Left	CustomerSystem	Seating, Relationships, CafeGram, Events
RelationshipChanged	RelationshipSystem	UI, Memory, DialogueSnippet, RegularStatus
DecorChanged	DecorationSystem	CafeIdentity, Attraction, Save
PostPublished / TrendStarted	CafeGram	Reputation, Demand, Phone Notifications
TransactionPosted	Economy	Bank UI, ShiftSummary, Analytics
MilestoneUnlocked	Progression	Shop, Memory, Events, Ownership
WeatherChanged	Weather	Ambience, CustomerSpawner, EventSystem

4. Modular System Specification

Reading each system

Purpose = why it exists. Owns = the only data it may author. Inputs = events/services it reads. Outputs = events/state it exposes. Dependencies = allowed hard dependencies. Phase = MVP / Core / Stretch.

4.1 Interaction, Coffee & Service

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Interaction System	Ray/volume-based context interaction; unified prompts and hold/tap timing.	Current interactable, interaction context	Input; scene interactables	InteractionStarted/Completed/Cancelled	Input only	MVP
Coffee Crafting	Runs tactile coffee actions and validates completed drink composition.	Active craft session, cup contents, step state	Recipe defs; interact actions	DrinkCrafted	Interaction, Recipe	MVP
Recipe System	Defines required ingredients/actions/tolerances and unlock rules.	Known recipes/unlocks	Progression; ingredient defs	RecipeUnlocked	Progression	MVP
Ingredient System	Tracks ingredient types, portions and machine compatibility.	Runtime ingredient quantities in stations	Inventory	IngredientConsumed/Added	Inventory	MVP
Order System	Creates and tracks customer orders from request to completion/expiry.	Order lifecycle + requested items	Customer request; drink result	OrderCreated/Completed/Failed	Customer, Coffee	MVP
Tray Serving	Allows placing drink/food on physical tray and delivering as a set.	Tray slots + carried items	Interact; completed items	TrayDelivered	Interaction, Order	Core
Station System	Reusable state model for grinder, espresso machine, milk steamer, sink, display.	Station readiness, local queues	Interaction; upgrades	StationStateChanged	Interaction	MVP
Food/Pastry Service	Simple pick/plate/serve items; no cooking simulation.	Display stock	Inventory; order	FoodServed	Order, Inventory	Core

Coffee Crafting Contract

- Every action is 1-3 seconds and readable from animation + audio; realism is subordinate to feel.
- A drink is represented by DrinkBuild: cup + base + milk + syrups/toppings + quality metadata.
- Recipe validation returns success, mismatch reasons, and quality score; it never directly pays money.
- The Order System decides whether a crafted item satisfies an order.
- Equipment upgrades change speed, tolerance, capacity or presentation - not the core input language.

4.2 Customers & Social Life

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Customer Director	Controls who may spawn and when; respects capacity, weather, reputation and attraction.	Spawn schedule + active count	Shift, Attraction, Weather	CustomerSpawnRequested	Shift, Attraction	MVP
Customer Agent FSM	Runs enter -> queue -> order -> wait -> seat -> consume -> activity -> leave.	Per-agent transient state	Order, seating, nav, events	CustomerStateChanged	Order, Seating	MVP
Customer Profile	Persistent identity: name, archetype, preferences, usuals, posting traits, schedule bias.	Definition references + persistent relationship key	Static profile SO	Profile queried	None	MVP
Seating System	Reserves, assigns and releases valid seats/tables.	Seat occupancy/reservations	Customer needs; decor layout	SeatReserved/Released	Decor	MVP
Relationship System	Tracks familiarity	XP/tier, known usual,	Visits, service quality	RelationshipChanged/	Customer, Order	Core

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
	with each recurring customer.	story beat index		RegularUnlocked		
Preference System	Scores drinks, styles, seats, music and events per profile.	No persistent truth beyond cached scores	Customer profile + cafe state	PreferenceEvaluated	Identity, Music	Core
Customer Activity System	Runs low-cost ambient activities: phone, laptop, read, chat, window, pet cat.	Current activity	Seat tags; profile	ActivityStarted/Ended	Seating	Core
Story Snippet System	Delivers short authored life snippets gated by relationship/milestone s.	Seen snippet IDs	Relationship, event context	StorySnippetShown	Relationship	Stretch

4.3 Phone, CafeGram & Organic Discovery

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Phone Shell	Diegetic hub for Bank, CafeGram, Shop, Manager, Messages and stats.	Open app/navigation state	Input; notifications	PhoneOpened/ AppChanged	UI services	Core
CafeGram	Generates customer posts from actual visits, decor, drinks and events.	Post feed, likes/comments counters, followers	Customer visit context, photo anchors	PostPublished	Customers, Identity	Core
Photo Capture	Captures/frames lightweight in-world screenshots from tagged camera points when needed.	Captured image references	CafeGram trigger	PhotoCaptured	Rendering	Stretch
Trend/Demand	Turns high-performing posts or events into temporary demand modifiers.	Active trends + expiry	CafeGram, events	TrendStarted/Ended	Order Director	Core
Notification	Queues non-intrusive phone alerts and badges.	Unread counts/notification queue	Any domain event	NotificationRaised	Phone	Core
Cafe Manager App	Displays menu, prices, opening settings and simple business controls.	Player-selected menu/settings	Recipes, economy	MenuChanged	Recipe, Economy	Core

Social Media Rule

Do not use live generative AI in v1. Posts are template-driven but context-aware: customer personality + actual drink + seat + weather + decor/event tags. It should feel personal while remaining deterministic, local, cheap and safe.

4.4 Decor, Identity & Attraction

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Decoration Placement	Place/move/remove owned decor with grid/socket/free-placement rules.	Placed decor transforms + IDs	Inventory/Shop	DecorChanged	Save	Core
Decor Definition	Defines cost, style tags, functional tags, footprint, unlock tier.	Static SO data	None	Queried by systems	None	MVP
Cafe Identity	Aggregates style/function scores from placed decor,	Current identity vector	Decor/menu/music	IdentityChanged	Decoration	Core

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
	menu, music and upgrades.					
Attraction System	Converts identity + reputation + trends into visitor archetype weights.	Current attraction weights	Identity, reputation, trends	AttractionChanged	Identity	Core
Photo Spot System	Marks highly photogenic arrangements/anchors for CafeGram opportunities.	Candidate photo spots	Decor layout + tags	PhotoOpportunity	Decoration	Stretch
Expansion/Renovation	Unlocks fixed structural upgrades: extra seating, wall finish, window bar, terrace/room.	Purchased expansions	Economy/progression	CafeLayoutChanged	Economy	Stretch

4.5 Economy, Progression & Ownership

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Economy Ledger	Single authoritative source for money transactions.	Balance + transaction log	Sales, purchases, rewards	TransactionPosted	None	MVP
Pricing	Computes item prices, tips and optional player menu price bands.	Menu price settings	Recipes, upgrades	PriceChanged	Economy	Core
Reputation	Tracks public cafe reputation separately from individual relationships.	Reputation XP/tier	Orders, posts, events	ReputationChanged	CafeGram, Orders	Core
Progression	Central unlock graph for recipes, shop items, robots, events and decor tiers.	Unlocked content/milestones	Money, reputation, relationships	MilestoneUnlocked	Economy, Rep	MVP
Shop/Catalog	Lists purchasable equipment, ingredients, decor, music and upgrades.	Owned purchase state	Progression, economy	PurchaseCompleted	Economy	Core
Ownership Campaign	Tracks requirements to purchase cafe and resolves campaign completion.	Ownership progress + purchased flag + cafe name	Economy, reputation, milestones	CafePurchased	Progression	Core
Collections	Tracks recipes, records, memories, cups, milestones and completion.	Collected IDs	Progression/events	CollectionUpdated	Save	Stretch

4.6 Chores, Robots & Cafe Operations

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Cleaning	Creates spills/dirty tables/cups and resolves them through simple interactions.	Dirty state per cleanable	Customer usage/events	Cleaned	Interaction	Core
Inventory	Tracks consumable stock and owned placeable items.	Ingredient counts, owned item IDs	Purchases, consumption	InventoryChanged	Save	MVP
Restocking	Moves purchased ingredients from storage/delivery into	Pending deliveries/local storage	Inventory, shift	RestockCompleted	Inventory	Core

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
	usable stations.					
Cleaner Robot - Mochi	Autonomously cleans floor spills/crumb targets; never cleans stations or makes coffee.	Robot state, battery/availability optional	Cleaning targets	TargetCleaned	Cleaning	Core
Delivery Robot	Carries completed tray/item from pickup point to assigned table; optional assist.	Robot queue/state	Order ready, table target	DeliveryCompleted	Orders, Nav	Stretch
Robot Cosmetics	Optional skins/stickers/accessories for personality and collection.	Equipped cosmetic IDs	Shop/collections	CosmeticChanged	Robots	Stretch

Automation Rule

Robots automate low-value chores only. The player remains the barista. Core coffee craft must never disappear behind automation.

4.7 Shift, Weather, Seasons & World

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Shift State Machine	Controls PreOpen -> Open -> LastCall -> Closed -> Summary -> Progress.	Current shift state/time	Player start/close, clock	ShiftOpened/Closed	Save	MVP
Game Clock	Advances shift time and exposes normalized day/time.	Current day/time	Shift speed	TimeChanged	Shift	MVP
Weather	Selects rain intensity, storm, clear, snow etc. and publishes ambience parameters.	Current weather state	Day seed/events	WeatherChanged	Audio/VFX	Core
Season	Provides longer-term visual/content modifier for autumn/winter/spring.	Current season/day index	Campaign progression	SeasonChanged	Weather	Stretch
Rare Event Director	Schedules special nights and micro-events with cooldowns/conditions.	Active event + history	Weather, reputation, day	EventStarted/Ended	Shift	Core
Neighborhood Evolution	Swaps pre-authored exterior states as cafe milestones progress.	Neighborhood stage	Ownership/progression	NeighborhoodChanged	Progression	Stretch
Ambient Street	Controls traffic, pedestrians, window light sweeps, exterior loops.	Ambient spawn state	Weather/time	None	World only	Core

4.8 Cat, Memories, Music & Emotional Systems

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Cafe Cat - Milo	Persistent ambient companion unlocked through a small stray-cat arc.	Cat unlock, location, simple state	Shift, customer activity	CatStateChanged	Save	Core
Memory Wall	Creates authored Polaroid/memory entries for meaningful milestones.	Unlocked memory IDs + display slots	Milestones, relationships, events	MemoryUnlocked	Progression	Core

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Music/Record Player	Lets player choose unlocked tracks/playlists; exposes music tags to preferences.	Selected track + unlocked records	Phone/record player	MusicChanged	Audio	Core
Signature Drink Creator	Allows combining bounded ingredients/toppings, validating recipe viability, naming it, and adding it to menu.	Custom drink definitions in save	Unlocked ingredients	SignatureDrinkCreated	Recipe, Menu	Stretch
Closing Reflection	Quiet post-shift summary and free movement state; highlights earnings, posts, regulars and memories.	Summary snapshot	Shift close	SummaryReady	Economy, Social	Core

4.9 Cross-Cutting & Platform Systems

System	Purpose	Owns	Inputs	Outputs	Deps	Phase
Save/Load	Versioned persistent snapshot with per-module save payloads; autosave at safe checkpoints.	SaveGame root + versions	ISaveable modules	SaveCompleted/Loaded	All persistent modules	MVP
Settings/Input	Keyboard/mouse + gamepad rebinding, sensitivity, audio, graphics.	Settings profile	Input System	SettingsChanged	UI	MVP
UI/HUD	Minimal diegetic prompts, order cards, interaction feedback, phone, summaries.	Presentation state only	Domain events	User commands	All domains	MVP
Audio	Layered ambience, machine one-shots, foley, UI, music ducking, interior/exterior mix.	Mixer/state only	Domain events	None	All domains	MVP
Juice/Feedback	Central helpers for micro camera impulses, particles, haptics, highlights, number pops.	Transient feedback requests	Domain events	None	Presentation	Core
Tutorial/Onboarding	Contextual one-action-at-a-time teaching during first shift.	Seen tutorial flags	Progression/context	TutorialShown	UI	Core
Analytics/Telemetry	Local/dev + optional release analytics for funnel, economy, failure and timing.	Event records	Domain events	Logs/metrics	EventBus	Core
Debug Tools	Spawn customer/order, set money/reputation/weather/time, unlock content, inspect state.	Dev-only state	Services	Debug commands	All systems	MVP
Steam Platform	Achievements, cloud save readiness, rich presence optional, build/depot config.	Platform IDs/status	Progression/save	AchievementUnlocked	Platform SDK	Core

5. Recommended Core Data Contracts

Data Type	Minimum Fields
DrinkDefinitionSO	id, displayName, requiredSteps, ingredient rules, cup type, base price, tags, unlock rule
DrinkBuild	cupId, ingredients[], preparationSteps[], quality, creatorId?, customName?
OrderData	orderId, customerId, requestedItems[], createdAt, patience, status, quality result
CustomerProfileSO	id, name, archetype, schedule bias, drink likes, style likes, seat likes, posting traits, story refs
CustomerPersistentState	relationshipXP, tier, visits, knownUsual, seenStories[], lastVisitDay
DecorItemSO	id, prefab, cost, footprint, styleTags{}, functionTags{}, attractionModifiers{}, unlockRule
CafeState	placedDecor[], identityVector{}, reputation, menu, expansions, cafeName, ownershipFlag
SocialPostData	postId, customerId, trigger, referencedDrink/decor/event, captionTemplate, imageRef?, metrics, day
EconomyTransaction	id, amount, category, sourceId, day, timestamp
WorldState	day, time, weather, season, neighborhoodStage, activeEventId
SaveGame	schemaVersion, player/economy, cafe, customers, progression, collections, world, settingsRef

6. Dependency Rules

- No system writes another system's state directly. Request actions through methods/interfaces or publish commands.
- Economy is the only module that changes money. Progression is the only module that changes unlock state.
- Customer Agent owns transient behavior; Relationship System owns persistent familiarity.
- Coffee Crafting produces a DrinkBuild; Order System decides whether it satisfies an order.
- Decoration owns placement; Cafe Identity only reads decor tags and publishes aggregate scores.
- CafeGram reads visit context and produces posts; it does not directly spawn customers. Trend/Attraction modify spawn weights.
- UI never contains game rules. It binds to read models/presenters and sends commands.
- Save/Load never knows internal rules; modules serialize/deserialize their own payloads.

6.1 Suggested Assembly

Composition Root

Create one bootstrap scene/prefab that constructs services, loads save data, registers modules, then loads the cafe scene. Scene-level presenters resolve services through explicit references or injection. Avoid static global access wherever practical.

7. Build Plan - Prove Fun Before Scale

7.1 Vertical Slice Definition

The first playable

One small cafe. One espresso machine. Three drinks. Three recurring customers. Four seats. One rainy evening. Manual order -> craft -> serve -> sit -> leave loop. Basic money. Basic save. Strong sound/VFX. If this is not fun, do not add more systems.

Phase	Deliverable
P0 - Foundation	Bootstrap, EventBus, save schema, input, interaction, debug panel, ScriptableObject data conventions.
P1 - Coffee Slice	Stations, ingredients, 3 recipes, DrinkBuild, orders, quality validation, core audio/feedback.
P2 - Customer Slice	3 profiles, customer FSM, queue, seating, consume, leave, basic relationship persistence.
P3 - Shift & Economy	Clock, open/close flow, ledger, shift summary, basic shop/restock.
P4 - Atmosphere	Rain/night exterior, interior/exterior audio mix, traffic ambience, music, lighting pass.
P5 - Identity	Decor placement, decor tags, cafe identity, attraction-weighted customer selection.
P6 - Social	Phone shell, CafeGram posts, notifications, follower/reputation feedback, simple trends.
P7 - Progression	Unlock graph, robot cleaner, Milo, memory wall, ownership campaign goal.
P8 - Content/Polish	8-12 drinks, 12-20 customers, 20-40 decor items, rare events, achievements, tutorial, optimization.
P9 - Release	Steam build/depot, crash testing, save migration test, controller pass, QA, trailer-ready capture.

7.2 MVP vs Later

MVP / Must Ship	Later / Only After Core Is Fun
Manual coffee craft + stations	Signature drink creator
Orders + 3-5 drinks	Photo capture from NPC viewpoint
Recurring customers + seating	Deep story snippets
Basic relationship tiers	Delivery robot
Shift + economy + save	Seasons beyond one launch season
Rain/night ambience	Neighborhood evolution
Basic decor + identity tags	Expanded renovations/terrace
Phone shell + simple CafeGram	Complex trend chains
Cleaner robot	Robot cosmetics
Ownership goal	Large collections/endless completion

8. Design & Balance Rules

- The player should spend most active time making/serving coffee, not navigating menus.

- A standard drink should take roughly 10-20 seconds once learned.
- Customer patience creates light pressure, not punishment. Failure reduces tips or causes departure; it should not spiral the run.
- Each upgrade must either improve feel, create a new option, reduce a chore, or visibly transform the cafe.
- Every major progression tier should introduce one new interaction or emotional beat - not only larger numbers.
- Keep the cafe physically compact. Density increases coziness and makes decoration readable.
- No system should exist only because other simulator games have it.
- Prefer authored small moments over broad procedural complexity when both cost the same.

9. Presentation Direction

Area	Direction
Visual	Stylized cozy realism; Korean-inspired night cafe; warm wood/cream interior against cool rainy street.
Lighting	Warm practical lamps, soft emissive signage, wet exterior reflections, headlight sweeps, strong window contrast.
UI	Minimal, clean, modern mobile-app language; no heavy simulator panels. Phone is primary management surface.
Animation	Short tactile interaction clips; customers use small reusable ambient loops. Avoid animation-heavy cinematics.
Audio	Coffee machine detail, porcelain, door chime, rain layers, street wash, soft crowd, music with interior/exterior filtering.
Juice	Tiny camera impulse, haptics, steam/foam particles, object placement sounds, clean progress cues; never noisy or arcade-like.
Asset Use	Unify third-party assets with shared materials, palette, decals, lighting, signage, props and custom UI so the game does not read as an asset pack.

10. System Acceptance Checklist

Definition of Done for a Module

A system is not complete because it works once. It is complete when its public contract is clear, it has no hidden scene dependency, it serializes correctly if persistent, it exposes debug controls, and at least one isolated test/demo proves its main states.

- Single responsibility and named owner for each piece of state.
- ScriptableObject definitions separated from runtime state.
- Public interface/events documented in code.
- No direct writes into another domain module.
- Happy path + failure/cancel path implemented.
- Save/load test if persistent.
- Debug controls available for fast iteration.
- Profiler check for per-frame allocations / unnecessary Update loops.
- Basic unit/play-mode test or isolated test scene where useful.
- Presentation feedback attached only after logic is stable.

11. Antigravity Handoff Rules

How to use this GDD with an AI coding assistant

Implement one module at a time. Before coding, require a short technical plan containing: responsibility, owned data, public API/events, dependencies, save payload, debug hooks and test scenario. Reject solutions that couple unrelated modules or bury game rules inside UI/MonoBehaviour presentation code.

11.1 Required Output Per Module

- Folder/file layout.
- Interfaces and domain events first.
- ScriptableObject definition types.
- Runtime model/service/controller implementation.
- Presenter/MonoBehaviour adapters only where scene interaction is needed.
- Save payload and migration note if persistent.
- Debug commands/inspector utilities.
- Minimal test scene or reproducible test steps.
- Integration notes listing only allowed dependencies.

12. First Antigravity Implementation Brief

Start with Foundation - not gameplay content

Create the project architecture for P0 only: composition root/bootstrap, typed EventBus, versioned SaveGame shell, input wrapper, generic IInteractable contract, ScriptableObject ID conventions, debug command panel, and one isolated test scene. Do not implement coffee, customers, economy, social media or progression yet.

- Return the proposed folder structure and dependency graph before writing implementation code.
- Use assembly definitions or clear feature folders so domains can remain independent.
- Every persistent model uses stable string IDs; never persist direct Unity object references.
- Add a simple debug window that can print registered services, publish test events and save/load a dummy payload.
- Finish P0 with a clean test: boot -> register -> interact with dummy object -> publish event -> save -> reload -> verify restored dummy state.

LOCKED CORE: manual coffee craft + living regulars + meaningful decor + organic social discovery + cozy night atmosphere + visible long-term ownership journey.

Anything new must strengthen one of those six things or stay out of v1.